

**PRACTICE EXERCISES OF THE MICROPROCESSORS
& MICROCONTROLLERS**

Instructor: The Tung Than

Student's name: Gia Hung Nguyen

Student code: 24520603

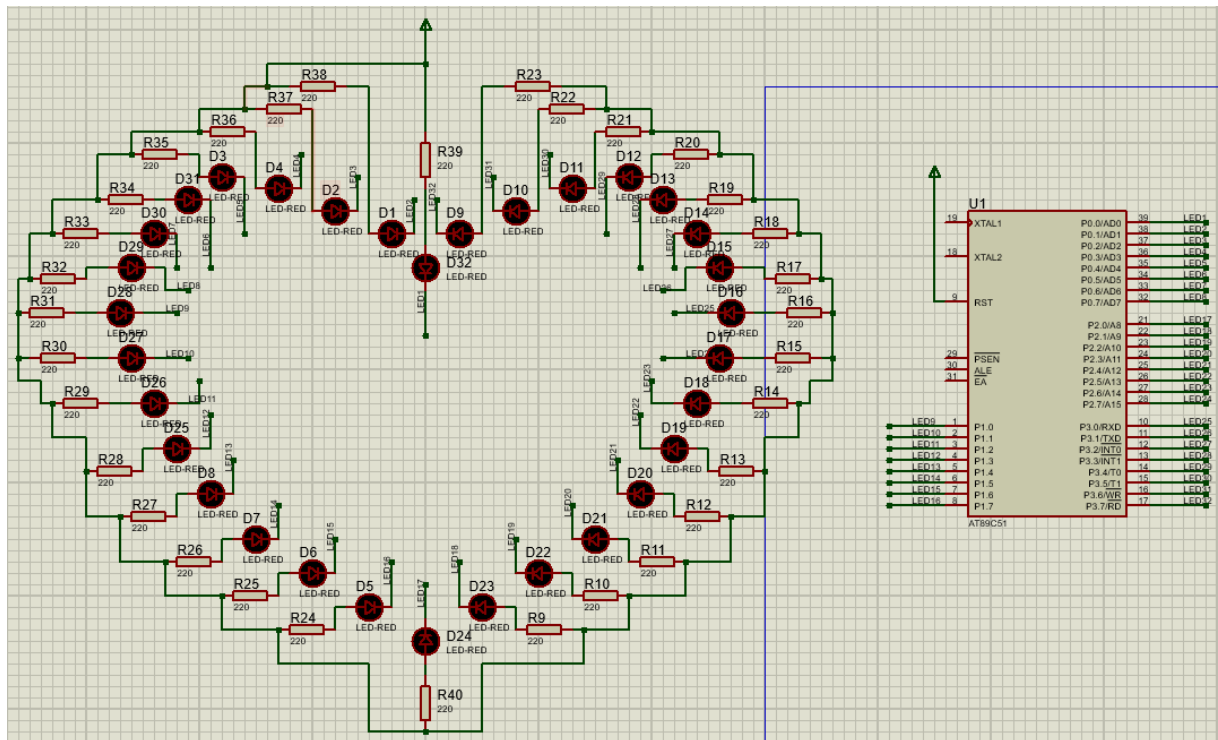
PRACTICE REPORT NO 1
ACQUAINTANCE WITH PROTEUS AND THE
8051 MICROCONTROLLER FAMILY

- I. Thiết kế**
- II. Xây dựng chương trình Assembly**
- III. Các bước thực hiện mạch in**

I. Thiết kế

Mô phỏng được thực hiện trên phần mềm Proteus với vi điều khiển họ 8051 (AT89C51). Các thành phần chính trong mạch bao gồm:

- Khối vi điều khiển trung tâm: IC AT89C51.
- 32 LED hiển thị: 32 LED đơn được kết nối với 4 Port của vi điều khiển. Trong thiết kế này, LED được mắc theo kiểu Active-Low (Anode nối nguồn VCC, Cathode nối với chân vi điều khiển).



Hình 1: Sơ đồ mạch 32 LED trái tim trên Proteus

II. Xây dựng chương trình Assembly

1. Hiệu ứng và nguyên lý hoạt động

Mạch LED được kết nối theo cấu hình **tích cực mức thấp (Active-Low)**: cực Anode nối với nguồn ($VCC = 5V$) và cực Cathode nối vào Port của vi điều khiển. Do đó:

- **Xuất mức logic 0 (0V):** Tạo sự chênh lệch điện áp, dòng điện chạy qua làm **LED sáng**.
- **Xuất mức logic 1 (5V):** Điện áp ở hai đầu bằng nhau, không có dòng điện nên **LED tắt**.

Chương trình gồm có 3 hiệu ứng:

- **Hiệu ứng 1:** Chớp nháy luân phiên các LED chẵn và lẻ.
- **Hiệu ứng 2:** Sáng đơn từng đèn chạy tuần tự qua các Port.
- **Hiệu ứng 3:** Sáng đuổi 5 đèn chạy dịch vòng qua toàn bộ 32 LED.

ASSEMBLY	GIẢI THÍCH
ORG 0000H JMP MAIN ORG 0100H	- ORG 0000H: Bắt đầu chương trình tại địa chỉ 0000H. - JMP MAIN: Nhảy đến MAIN để bỏ qua vùng nhớ ngắt. - ORG 0100H: Đặt vùng nhớ cho chương trình bắt đầu từ 0100H.
MAIN: CALL EFFECT_1 CALL EFFECT_2 CALL EFFECT_3 JMP MAIN	Chương trình chính: Gọi tuần tự 3 chương trình con tương ứng với 3 hiệu ứng. Lệnh JMP MAIN tạo vòng lặp vô tận để mạch chạy liên tục.
DELAY1: MOV R0, #192 LOOP1: MOV R1, #192 CONT: DJNZ R1, LOOP2 DJNZ R0, LOOP1 RET LOOP2: NOP JMP CONT RET	Chương trình con tạo độ trễ 1: Dừng cho hiệu ứng 1. Sử dụng thanh ghi R0, R1 làm vòng lặp đếm lùi (DJNZ) để tạo độ trễ. Chu kì máy = $12/12 = 1 \mu s = 10^{-3} \text{ ms}$. Thời gian trễ: $192 \times 192 \times 5 \times 10^{-3} \text{ ms} \approx 184.32 \text{ ms}$. Lệnh NOP tốn 1 chu kỳ máy kết hợp lệnh nhảy để tiêu tốn thêm thời gian trong vòng lặp con.

ASSEMBLY	GIẢI THÍCH
DELAY2: MOV R2, #120 LOOP3: MOV R3, #130 CONT1: DJNZ R3, LOOP4 DJNZ R2, LOOP3 RET LOOP4: NOP JMP CONT1 RET	Chương trình con tạo độ trễ 2: Dùng cho hiệu ứng 2 và 3. Tương tự DELAY1, dùng R2, R3 để tạo vòng trễ có thời gian ngắn hơn giúp LED quét mượt mà. Thời gian trễ: $120 \times 130 \times 5 \times 10^{-3} \text{ ms} = 78 \text{ ms}.$
EFFECT_1: MOV R4, #13 E1_LOOP: MOV P0, #55H MOV P1, #55H MOV P2, #55H MOV P3, #55H CALL DELAY1 MOV P0, #0AAH MOV P1, #0AAH MOV P2, #0AAH MOV P3, #0AAH CALL DELAY1 DJNZ R4, E1_LOOP RET	Hiệu ứng 1: Chớp nháy luân phiên các LED chẵn và lẻ - MOV R4, #13: Thiết lập số lần lặp là 13. - MOV Px #55H: Làm sáng các LED lẻ (01010101B). - CALL DELAY1: Gọi hàm DELAY1 tạo độ trễ. - MOV Px #0AAH: Đảo ngược lại, làm sáng các LED chẵn (10101010B). - CALL DELAY1: Gọi hàm DELAY1 tạo độ trễ. - DJNZ R4, E1_LOOP: Mỗi vòng lặp giảm R4 đi 1, nếu R4 khác 0 thì quay lại nhãn E1_LOOP. Tổng thời gian thực thi: $13 \times 2 \times 184.32 \text{ ms} \approx 4.79 \text{ s}.$

ASSEMBLY	GIẢI THÍCH
EFFECT_2: MOV R4, #2 E2_LOOP: MOV P1, #0FFH MOV P2, #0FFH MOV P3, #0FFH MOV P0, #07FH MOV A, P0 MOV R5, #8	Hiệu ứng 2: Sáng đơn từng đèn chạy tuần tự qua các Port - MOV R4, #2: Khởi tạo biến đếm R4 = 2 để chạy tổng cộng 2 vòng. - MOV P1/2/3, #0FFH: Xuất mức 1 để tắt toàn bộ LED ở Port 1, 2, 3. - MOV P0, #07FH: Bật điểm sáng đầu tiên ở LED 1 của Port 0 (01111111B). - MOV A, P0: Đưa giá trị của Port 0 vào thanh ghi A để chuẩn bị dịch. - MOV R5, #8: Khởi tạo biến đếm R5 = 8 (Vì mỗi PORT điều khiển 8 LED).
LOOP_P0: RL A MOV P0, A CALL DELAY2 DJNZ R5, LOOP_P0	- RL A: Quay trái thanh ghi A 1 bit (dịch bit 0 sang trái). - MOV P0, A: Xuất giá trị vừa dịch ra Port 0 để làm sáng LED tiếp theo. - CALL DELAY2: Gọi hàm trễ để nhìn thấy trạng thái LED. - DJNZ R5, LOOP_P0: Giảm R5 đi 1, nếu khác 0 thì nhảy về LOOP_P0.
MOV P0, #0FFH MOV P2, #0FFH MOV P3, #0FFH MOV P1, #07FH MOV A, P1 MOV R5, #8 LOOP_P1: RL A MOV P1, A CALL DELAY2 DJNZ R5, LOOP_P1	Thực hiện các lệnh tương tự như LOOP_P0 để quét sáng tuần tự 8 bóng LED của Port 1 . Tắt P0, P2, P3 và bật điểm sáng đầu tiên ở P1.

ASSEMBLY	GIẢI THÍCH
<pre> MOV P0, #0FFH MOV P1, #0FFH MOV P3, #0FFH MOV P2, #07FH MOV A, P2 MOV R5, #8 LOOP_P2: RL A MOV P2, A CALL DELAY2 DJNZ R5, LOOP_P2 </pre>	Thực hiện các lệnh tương tự như LOOP_P0 để quét sáng tuần tự 8 bóng LED của Port 2. Tắt P0, P1, P3 và bật điểm sáng đầu tiên ở P2.
<pre> MOV P0, #0FFH MOV P1, #0FFH MOV P2, #0FFH MOV P3, #07FH MOV A, P3 MOV R5, #8 LOOP_P3: RL A MOV P3, A CALL DELAY2 DJNZ R5, LOOP_P3 DJNZ R4, E2_LOOP RET </pre>	Thực hiện các lệnh tương tự như LOOP_P0 để quét sáng tuần tự 8 bóng LED của Port 3. Tắt P0, P1, P2 và bật điểm sáng đầu tiên ở P3. - DJNZ R4, E2_LOOP: Giảm R4 đi 1, nếu khác 0 thì lặp lại toàn bộ quy trình. Tổng thời gian thực thi: $2 \text{ vòng} \times 4 \times 8 \text{ bước} \times 78 \text{ ms} = 64 \times 78 \text{ ms} \approx 4.99 \text{ s}.$
<pre> EFFECT_3: MOV R4, #0E0H MOV R5, #0FFH MOV R6, #0FFH MOV R7, #0FFH MOV R0, #64 </pre>	Hiệu ứng 3: Sáng đuôi 5 đèn chạy dịch vòng qua toàn bộ 32 LED - MOV R4, #0E0H: Nạp trạng thái (11100000B) để bật sáng 5 LED tại Port 0. - MOV R5/R6/R7, #0FFH: Tắt hoàn toàn các LED ở Port 1, Port 2, Port 3. - MOV R0, #64: Khởi tạo biến đếm lặp 64 lần (để 5 LED chạy quanh trái tim đúng 2 vòng).

ASSEMBLY	GIẢI THÍCH
E3_LOOP: MOV P0, R4 MOV P1, R5 MOV P2, R6 MOV P3, R7 CALL DELAY2	- MOV Px, Rx: Lấy dữ liệu hiện tại từ 4 thanh ghi R4, R5, R6, R7 xuất ra 4 Port tương ứng để hiển thị đèn sáng lên LED. - CALL DELAY2: Gọi hàm tạo trễ để quan sát đèn sáng.
MOV A, R7 RLC A MOV A, R4 RLC A MOV R4, A MOV A, R5 RLC A MOV R5, A	- MOV A, R7: Đưa giá trị của thanh ghi R7 (Port 3) vào thanh ghi A để chuẩn bị dịch. - RLC A: Dịch bit cao nhất (Bit 7) ra và lưu tạm vào Cờ nhớ (Carry). Bit này sẽ được nối vòng về lại đầu Port 0. - MOV A, R4: Đưa giá trị R4 vào thanh ghi A. - RLC A: Cờ Carry (vừa lấy từ R7) chui vào bit thấp nhất (Bit 0) của A, đẩy bit cao nhất (Bit 7) của A ra Carry. - MOV R4, A: Lưu lại giá trị đã dịch xong vào R4. - MOV A, R5: Đưa giá trị R5 vào thanh ghi A. - RLC A: Lấy Cờ Carry (vừa nhận từ R4) dịch tiếp vào bit 0 của A, đẩy bit cao nhất ra Carry. - MOV R5, A: Lưu lại giá trị đã dịch xong vào R5.

ASSEMBLY	GIẢI THÍCH
MOV A, R6 RLC A MOV R6, A MOV A, R7 RLC A MOV R7, A DJNZ R0, E3_LOOP RET END	- MOV A, R6: Đưa giá trị R6 vào thanh ghi A. - RLC A: Lấy Cờ Carry (từ R5) dịch vào bit 0 của A, đẩy bit cao nhất ra Carry. - MOV R6, A: Lưu lại giá trị đã dịch xong vào R6. - MOV A, R7: Đưa giá trị R7 vào thanh ghi A. - RLC A: Lấy Cờ Carry (từ R6) dịch vào bit 0 của A, đẩy bit cao nhất ra Carry. Lúc này, chuỗi 32 bit đã hoàn thành dịch 1 vị trí. - MOV R7, A: Lưu lại giá trị đã dịch xong vào R7. - DJNZ R0, E3_LOOP: Giảm R0 đi 1, lặp lại cho đến khi đủ 64 bước. Tổng thời gian thực thi: $64 \times 78 \text{ ms} \approx 4.99 \text{ s}$.

2. Mô phỏng LED trên Proteus

Link video mô phỏng và File thiết kế: https://drive.google.com/drive/folders/1BvV14WYeG1gyec_xSWyArKJ3ZbXJaNv?usp=sharing

III. Các bước thực hiện mạch in

B1. Vẽ sơ đồ nguyên lý (Schematic Capture): Lựa chọn và kết nối linh kiện trên phần mềm (như Proteus, Altium) để định hình logic hoạt động của mạch.

B2. Thiết kế sơ đồ mạch in (PCB Layout): Chuyển đổi sang bản vẽ mạch in thực tế.

- **Sắp xếp linh kiện (Placement):** Tối ưu vị trí để mạch gọn, giảm nhiễu và dễ đi dây.
- **Đi dây (Routing):** Kết nối các chân linh kiện hợp lý, hạn chế tối đa việc cắt chéo nhau.

B3. Thiết kế thông số: Đảm bảo các tiêu chuẩn vật lý và an toàn.

- Khoảng cách an toàn giữa các đường mạch để tránh chạm chập, phóng điện.
- Độ rộng đường dây đủ lớn để chịu được dòng điện định mức, tránh đứt hoặc cháy mạch.

B4. In mạch lên giấy Decal: Xuất file thiết kế và dùng máy in laser in lên mặt láng của giấy Decal nhiệt.

B5. Ủi mạch nhiệt: Áp bản in lên phíp đồng sạch. Dùng bàn ủi nhiệt độ cao miết đều tay để mực in bám chặt vào bề mặt đồng.

B6. Ăn mòn hóa học (Etching): Ngâm mạch vào dung dịch FeCl_3 để ăn mòn phần đồng hở, giữ lại đường mạch nằm dưới lớp mực.

B7. Vệ sinh và Phủ bảo vệ: Tẩy sạch lớp mực in bằng axeton hoặc giấy nhám. Phủ một lớp nhựa thông lỏng để chống oxi hóa đường đồng.

B8. Khoan mạch và Lắp ráp: Khoan lỗ tại các pad đồng (mũi 0.8mm - 1.2mm). Cắm linh kiện đúng chiều, hàn cố định bằng thiếc và cắt gọn chân thừa.